DOCUMENTAÇÃO DO SISTEMA

ConecTorno

Site de comunicação e acompanhamento de monitores
para uso com o servidor existente no Raspberry Pi

Objetivo
Explicar o funcionamento do site, seus recursos, a comunicação com a API e os cuidados de instalação.

Versão documentada: pacote somente do site (frontend), sem servidor próprio, porque o raspberry já existe um servidor

Documento elaborado com base nos arquivos do pacote ConecTorno fornecido.

1. Visão geral

O ConecTorno é uma interface web para o professor publicar materiais e acompanhar até seis monitores. O professor acessa um painel protegido por login; cada monitor abre uma tela pública específica, identificada por um número na URL.

Importante: o pacote documentado contém somente os arquivos do site — HTML, CSS, JavaScript, imagem e biblioteca PDF.js. Ele não inclui o servidor Python/Flask, banco de dados, autenticação do backend nem serviço para iniciar a aplicação. Portanto, o funcionamento depende do servidor que já está instalado no Raspberry Pi.

Objetivos principais

- Publicar um PDF ou uma imagem para exibição nos monitores.
- Acompanhar o estado dos monitores pelo painel do professor.
- Abrir a tela de um monitor específico por um endereço próprio.
- Adicionar observações relacionadas a um monitor e removê-las quando necessário.
- Gerenciar o acesso do professor por cadastro inicial, login, alteração e recuperação de senha.

2. Componentes do sistema

Componente	Função
HTML — static/index.html	Estrutura das telas de login, painel do professor e visualização dos monitores.
CSS — static/style.css	Aparência, organização dos elementos e adaptação visual.
JavaScript — static/app.js	Controla as ações da interface e envia requisições HTTP para a API do servidor.
PDF.js — static/pdfjs/	Biblioteca usada para apresentar documentos PDF no navegador.
Servidor já instalado no Raspberry Pi	Entrega os arquivos do site, executa a API, valida sessões e processa operações e arquivos enviados.

3. Como o sistema funciona

3.1 Primeiro acesso do professor

Ao abrir o endereço do Raspberry, o site consulta /api/auth/status para saber se existe uma configuração de acesso e se há uma sessão válida. No primeiro uso, o professor pode criar usuário e senha; depois disso, normalmente entra com suas credenciais.

3.2 Painel do professor

Depois do login, a interface consulta /api/state e apresenta o estado disponível no servidor. Pelo painel, o professor pode enviar um PDF ou uma imagem, publicar ou remover um projeto geral, selecionar monitores, limpar a publicação de um monitor e registrar ou apagar observações.

3.3 Tela de cada monitor

Cada monitor usa o mesmo site, mas com um identificador na URL. O JavaScript lê esse número e consulta /api/monitor/<n>/display para buscar o conteúdo que deve aparecer. Também envia periodicamente uma chamada de presença (heartbeat) para /api/monitor/<n>/heartbeat, permitindo ao servidor acompanhar a atividade do monitor.

Exemplos de endereço:

http://ENDERECO-DO-RASPBERRY:PORTA/?monitor=1

http://ENDERECO-DO-RASPBERRY:PORTA/?monitor=2

… até monitor=6

Substitua ENDERECO-DO-RASPBERRY e PORTA pelos valores reais usados atualmente na rede. Não abra o arquivo index.html diretamente e não use o Live Server do VS Code para a operação real: o site precisa ser servido pela mesma origem que disponibiliza a API.

3.4 Envio de arquivos

O professor escolhe um PDF ou uma imagem no formulário de publicação. O navegador envia o arquivo para /api/upload. O servidor precisa armazená-lo e disponibilizar o arquivo por um caminho acessível, normalmente sob /uploads/.... A tela de monitor usa os dados retornados pela API para apresentar o material.

3.5 Senhas e sessão

A interface possui formulários para criar o acesso inicial, entrar, sair, alterar a senha e solicitar recuperação com código mestre que está com a equipe que o criou. A segurança real depende do backend: validação das credenciais, armazenamento seguro das senhas, proteção da sessão e validação CSRF nas operações administrativas.

4. Rotas da API necessárias

O JavaScript usa caminhos relativos que começam com /api/. O servidor existente precisa implementar rotas equivalentes, aceitar os métodos corretos e devolver JSON no formato esperado pelo frontend.

Método e rota	Finalidade
GET /api/auth/status	Verificar configuração de acesso e estado da sessão.
POST /api/setup	Criar o usuário do primeiro acesso.
POST /api/login e /api/logout	Iniciar e encerrar a sessão do professor.
POST /api/change-password	Alterar a senha do professor autenticado.
POST /api/password/recover	Recuperar o acesso com o código mestre.
GET /api/state	Carregar o estado do painel, projetos e observações.
POST /api/upload	Receber PDF ou imagem para publicação.
POST /api/project/delete	Remover a publicação geral.
POST /api/monitor/<1-6>/project/reset	Limpar a publicação específica de um monitor.
POST /api/monitor/<1-6>/note	Criar uma observação para um monitor.

DELETE /api/monitor/<1-6>/note/<id>	Apagar uma observação identificada pelo ID.
POST /api/monitor/<1-6>/heartbeat	Informar a presença/atividade do monitor.
GET /api/monitor/<1-6>/display	Consultar o conteúdo destinado ao monitor.

5. Requisitos de compatibilidade

- A pasta static deve ser servida pelo mesmo servidor e endereço que respondem às rotas /api/....
- As rotas de monitor /heartbeat e /display devem aceitar chamadas sem token, pois esta interface não envia token de monitor.
- As operações do professor devem continuar protegidas pela sessão e pelo mecanismo CSRF exigido pelo backend.
- As respostas devem ser JSON e incluir os campos esperados pela interface, como ok, erro/mensagem e os dados do estado, projeto e observações.
- Os arquivos enviados e seus links devem continuar disponíveis no Raspberry, normalmente por /uploads/....
- Os seis identificadores válidos de monitor são de 1 a 6.

Atenção: esta documentação descreve o contrato esperado pelo site. Ela não confirma que o app.py atualmente instalado no Raspberry já implementa todas as rotas. A compatibilidade deve ser verificada antes da substituição dos arquivos.

6. Instalação segura no Raspberry Pi

1. Identificar a pasta do projeto. Localize a pasta usada pelo serviço atual do ConecTorno e confirme qual diretório static ele publica.

2. Fazer backup. Copie a pasta static atual e mantenha uma cópia dos arquivos de configuração, dados e uploads.

3. Conferir a API. Compare as rotas do servidor existente com a lista desta documentação. Não presuma compatibilidade apenas porque a página abre.

4. Substituir somente o site. Se a API for compatível, copie a pasta static deste pacote para o local esperado pelo servidor.

5. Preservar o backend. Não apague nem substitua app.py, configurações, dados, autenticação, chave secreta ou uploads.

6. Reiniciar o serviço existente. Use o procedimento normal de reinicialização do projeto no Raspberry; não inicie um segundo servidor para o mesmo site.

7. Testar professor e monitores. Abra o endereço habitual, faça login e teste publicação, exibição, observações e os monitores que serão usados.

7. Diagnóstico de problemas

Sintoma	Possível causa / verificação
404 em uma rota /api/...	A API do servidor não possui a rota esperada ou o caminho está diferente.
401 ou 403 na tela do monitor	O backend exige autenticação/token para monitor, ou bloqueia as rotas públicas de heartbeat/display.
Failed to fetch	Endereço ou porta incorretos, servidor indisponível, falha de rede ou bloqueio do navegador.
Login funciona, mas publicar/apagar falha	Verifique sessão, proteção CSRF, permissões e formato de resposta da rota.
PDF/imagem não aparece	Confira o resultado do upload, o caminho do arquivo, a rota de exibição e o acesso a /uploads/.
Site abre, mas os dados não atualizam	A página estática pode estar acessível enquanto a API está indisponível ou incompatível.

8. Segurança e manutenção

- Não compartilhe senhas nem códigos de recuperação.
- Use senhas fortes e altere credenciais padrão, caso existam.
- Mantenha o Raspberry e o servidor atualizados e restrinja o acesso à rede conforme a necessidade da escola/oficina.
- Faça backup antes de qualquer atualização; teste a restauração dos arquivos importantes.
- Não exponha a interface à internet sem revisar autenticação, autorização, CSRF, uploads e configuração de rede.
- Registre mudanças de versão e guarde a documentação junto do projeto.

9. Resumo

O ConecTorno desta versão é o frontend do sistema. Ele fornece a interface visual e faz requisições para uma API já existente. Para funcionar no Raspberry Pi sem criar outro servidor, o serviço atual precisa servir a pasta static e implementar as rotas, permissões e formatos de dados descritos neste documento. Se houver incompatibilidade, a solução é adaptar o frontend ou o backend existente de forma controlada, preservando dados e configurações.

Documento baseado no pacote ConecTorno_Raspberry_Somente_Site_Sem_Servidor.zip e no arquivo API_COMPATIBILIDADE.md incluído nele.